

Regression Algorithms for Learning & Its Application on Crypto Currency Markets

Interim Report

Syed Maesum Hussain Zaidi

Undergraduate Final Year Project

Supervised By : Xu Feng

Department of Computer Science

Royal Holloway, University of London

1. Abstract

This interim report presents the work completed so far in a project investigating how regression based learning methods can be used to model and forecast Bitcoin (BTC-USD) price movements. The project also explores how different learning models compare performance wise at this task. The report begins by building a detailed theoretical understanding of Ordinary Least Squares (OLS), Ridge Regression and Lasso Regression, focusing on their cost functions, closed-form solutions, and generalisation behaviour. This theory is complemented by a review of financial forecasting literature, which demonstrates the extent to which regression principles continue to be a part of modern machine learning approaches. The report then transitions into the practical phase which involves looking at the proof-of-concept regression model on synthetic dataset implementation to verify the behaviour of the Ridge estimator. Finally the report discusses the main program which is a full BTC-USD forecasting system incorporating engineered features, a walk-forward expanding-window validation framework, and from scratch implementations of OLS, Ridge and Lasso Regression. The system is capable of generating next-day BTC price predictions and evaluating model performance across a range of metrics. Together, these components provide a robust theoretical and computational foundation for the upcoming expansion of the modelling system to nearest-neighbour algorithms, neural networks, a graphical user interface, and multi-cryptocurrency support.

2. Introduction

Financial time-series prediction is challenging due to the presence of noise, structural changes and non-stationarity. Bitcoin, as the most dominant cryptocurrency, displays particularly volatile behaviour influenced by macroeconomic events, market sentiment and liquidity fluctuations. Despite these complexities, regression methods remain a vital starting point for understanding relationships within financial data because they offer interpretability, mathematical transparency and a baseline for more complex models.

The first phase of this project therefore concentrated on developing a deep understanding of regression algorithms and regularisation techniques. Ordinary Least Squares provides a foundation for modelling linear relationships, while Ridge Regression and Lasso Regression incorporate regularisation terms that help manage multicollinearity, instability and overfitting which are common issues in financial datasets. This theoretical groundwork was essential for designing practical forecasting systems later in the project.

Alongside this theoretical investigation, a literature review was conducted to explore how regression principles have previously been applied in financial forecasting research. The studies analysed highlight the continued relevance of linear modelling concepts even within advanced machine-learning architectures, as well as the importance of feature representation in predicting financial movements. These insights directly informed the feature-engineering and modelling strategies adopted in the project.

To connect theory with implementation, a proof-of-concept program was created using synthetic dataset. This allowed for the controlled study of Ridge Regression behaviour and provided verification that the Python implementation matched theoretical expectations. With this validation in place, the project progressed to the development of a BTC-USD prediction system. While still under works, this model demonstrates the feasibility of applying regression algorithms (OLS, Ridge and Lasso) to real cryptocurrency data and forms the foundation for further development through the implementation of other types of learning algorithms.

3. Aims & Objectives

The aim of this project is to investigate the effectiveness of regression algorithms, specifically Ordinary Least Squares (OLS), Ridge Regression, and Lasso Regression for forecasting the next-day price movement of the cryptocurrency pairs. The project seeks to understand both the mathematical foundations of these models and their practical behaviour when applied to highly volatile financial time-series data. Alongside this analytical goal, the project also aims to begin developing a structured, modular software framework that will later evolve into a full machine-learning forecasting system. To support this aim, the project establishes several objectives:

Aims

- To develop a clear understanding of OLS, Ridge, and Lasso regression, including their mathematical derivations and regularisation behaviour.
- To analyse the suitability of these models for financial forecasting, particularly in environments characterised by noise, volatility, and potential non-stationarity.
- To build a functioning regression-based forecasting pipeline capable of processing real BTC-USD data, engineering relevant features, training models, and generating predictions.
- To produce an initial object-oriented design that outlines the long-term structure of the full application, guiding future expansion.

Objectives

- To implement a proof-of-concept regression system using a synthetic dataset, allowing the theory of linear regression to be tested in a controlled setting.
- To collect and preprocess real BTC-USD historical data and generate a feature set appropriate for next-day forecasting.
- To implement OLS, Ridge, and Lasso regressions from first principles and integrate them into a coherent pipeline.
- To evaluate the performance of each model using metric-based analysis (e.g., MSE, R^2 , directional accuracy) and visualisation techniques.
- Identify limitations of the current linear modelling approach and prepare the groundwork for future extensions such as nearest-neighbour algorithms, neural networks, and a graphical user interface.

4. Background Theory

The theoretical basis for this project centres on three regression models: OLS, Ridge Regression and Lasso Regression. These models provide different approaches to estimating the relationship between a set of features and a continuous target variable.

The linear regression model assumes that the output variable can be expressed as a linear combination of input features, written in matrix form as:

$$y = X\beta + \varepsilon$$

with:

- y being a $N \times 1$ vector of outputs received
- X being a $N \times p$ design matrix containing input features
- β being a $p \times 1$ coefficient vector
- ε being a vector of random noise
- p is the number of input variables

The OLS estimator minimises the sum of squared residuals and leads to the closed-form solution:

$$\beta = (X^T X)^{-1} X^T y$$

- y = Vector of actual (observed) outputs
- X = Design matrix or feature matrix
- β = Vector of model parameters
- $()^T$ = Transpose operator
- $()^{-1}$ = Inverse

Although OLS is conceptually straightforward and computationally efficient, it becomes unstable when predictors are highly correlated or when the design matrix is ill-conditioned. These issues are particularly common in financial modelling, where technical indicators frequently overlap in the information they capture.

To address instability, Ridge Regression introduces an L2 penalty to the cost function, resulting in the estimator:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Where:

- λ = Regularization parameter
- I = Identity Matrix

This modification shrinks coefficients toward zero, improving numerical stability and reducing variance. Ridge Regression therefore provides a more robust foundation for modelling in noisy environments such as cryptocurrency markets.

Lasso Regression introduces an L1 penalty instead, which encourages sparsity in the coefficient vector. Unlike Ridge Regression, Lasso can force some coefficients exactly to zero, effectively performing feature selection. There are no closed-form solutions for lasso regression due to the absolute value term. Instead, interactive methods are used:

- Coordinate Descent: updates each coefficient one at a time using a soft-thresholding rule.
- LARS (Least Angle Regression): computes the entire path of solutions as λ decreases.

The implementation of Lasso used in this project is the Coordinate Descent method.

While Ridge Regression retains all predictors and stabilises their weights, Lasso helps identify the most influential variables. Both methods modify the bias-variance trade-off: regularisation increases bias slightly but reduces variance, improving generalisation on unseen data. These theoretical insights form the conceptual framework basis for the modelling work in the project.

5. Literature Review

The literature reviewed for this project provides insight into how regression principles continue to influence modern financial prediction techniques. Fischer and Krauss (2018) applied long short-term memory (LSTM) networks to forecast movements in the S&P 500 Index. Although LSTMs are significantly more complex than linear models, their internal computations rely on weighted linear transformations similar to those used in regression. Their results highlight the value of incorporating sequential structure into financial prediction, yet they also reinforce the importance of understanding linear models, as many core operations in deep learning extend regression concepts.

In a different approach, Patel, Shah, Thakkar, and Kotecha (2015) explored the impact of feature engineering on financial forecasting by testing several machine-learning algorithms on both continuous technical indicators and a trend-encoded dataset. Their findings demonstrated that the choice of input representation has a substantial effect on prediction performance. This outcome directly informs the modelling decisions in the present project, where feature engineering plays a central role in preparing BTC price data for regression.

Across both studies, a common theme emerges: regression principles remain foundational even as models grow more advanced. The emphasis on well-structured input features and the consistent use of linear transformations throughout machine learning models underscore the importance of a strong regression based foundation in financial prediction projects.

6. Summary of Early Deliverables Completed

As part of the initial project requirements, an overview of Ridge Regression was produced, explaining its mathematical formulation, the role of training and test sets, and the key components of its cost function and estimator. This early work provided a clear foundation for understanding how regularisation affects model stability and generalisation. In addition, hand-worked examples of Ridge Regression were developed to demonstrate practical understanding of matrix operations, coefficient computation and the influence of the regularisation parameter. These examples were verified using a prototype Python program, ensuring that the theoretical calculations aligned with computational results.

A proof-of-concept experiment was then implemented using a synthetic dataset designed to emulate a simple linear relationship with added noise. This allowed the behaviour of Ridge Regression to be examined in a controlled environment. The coefficient shrinkage observed with increasing regularisation matched theoretical expectations, and the predictions produced by the custom implementation aligned with those generated by standard machine-learning libraries, in this case scikit-learn library. This stage of the project strengthened confidence in both the mathematical understanding of regression and the correctness of the implementation, laying the foundation for transitioning to real BTC-USD data.

7. Proof of Concept Using Synthetic Data

Following the completion of the theoretical and literature foundations, a proof-of-concept model was developed to verify the correctness of the regression algorithms before applying them to real financial data. A synthetic dataset was generated to emulate a simple linear relationship with controlled noise, allowing clear observation of how Ridge Regression behaves as the regularisation parameter varies. This environment enabled a direct comparison between the analytical solution of Ridge Regression and the numerical output of the prototype Python implementation.

In the synthetic experiment, the design matrix was constructed with known input features and a predetermined coefficient vector. Gaussian noise was added to the targets to simulate real-world uncertainty. Ridge Regression was then applied for a range of regularisation strengths. As expected, increasing the value of λ reduced the magnitude of the coefficients, demonstrating the intended shrinkage effect. Moreover, the predictions produced by the custom Ridge implementation closely matched those obtained from the equivalent scikit-learn model, confirming that the custom linear algebra routines and loss calculations were functioning correctly. The experiment validated both the mathematical understanding and the computational implementation of the regression model, ensuring that the techniques were robust enough to be applied next to the more volatile BTC-USD dataset.

This synthetic validation served an important role: it isolated the model from market complexities and confirmed that any later discrepancies or unexpected outcomes in cryptocurrency modelling would arise from data properties rather than implementation errors. Having established this foundation, the project was then able to transition to the real-world BTC-USD dataset.

8. BTC-USD Modelling Methodology

Building on the synthetic proof-of-concept, the next stage of the project involved developing a full regression-based pipeline for forecasting Bitcoin's daily closing price. This pipeline was implemented entirely in Python without relying on inherited external models, ensuring full control over the learning process and full understanding of each computational step. The methodology reflects both practical considerations from financial modelling and the theoretical principles explored earlier.

8.1 Data Source & Preprocessing

The dataset consists of daily BTC-USD market data, including open, high, low, close and volume values. The goal of the model is to predict the next-day closing price using historical price behaviour and engineered technical indicators. Before feature construction, the raw data was processed to handle missing values, ensure temporal ordering and convert date labels into an index suitable for walk-forward evaluation.

Preprocessing also included calculation of daily returns, which provide a normalised representation of price movement. Returns help stabilise variance and improve interpretability, although the final target variable for forecasting was still the actual closing price.

8.2 Feature Engineering

A wide range of features was generated to capture the relevant dynamics of Bitcoin price movements. These included lagged prices, rolling statistics and technical indicators. For example, moving averages (MAs) were used to smooth short-term noise and highlight long-term trends. Exponential moving averages (EMAs) provided responsiveness to recent price behaviour. Momentum-based indicators, such as rate of change, helped quantify the strength and direction of recent market movements. Volatility indicators, including rolling standard deviation and true range values, were incorporated to help the model account for periods of high uncertainty.

Engineered features were selected to combine traditional technical-analysis insights with regression-ready numerical structure. Each feature was normalised based on the statistics of the training period to prevent information leakage. The feature matrix grew substantially as indicators and lags were added, reinforcing

the importance of using regularised models like Ridge and Lasso, which are better suited to handling multicollinearity than OLS.

8.3 Model Implementation

OLS, Ridge Regression and Lasso Regression were implemented manually using matrix operations and iterative optimization. OLS used the closed-form normal equation, while Ridge employed its regularised variant. Lasso required an iterative coordinate-descent solver to account for the non-differentiable absolute-value penalty. Implementing these models directly allowed close inspection of training dynamics, stability and numerical behaviour.

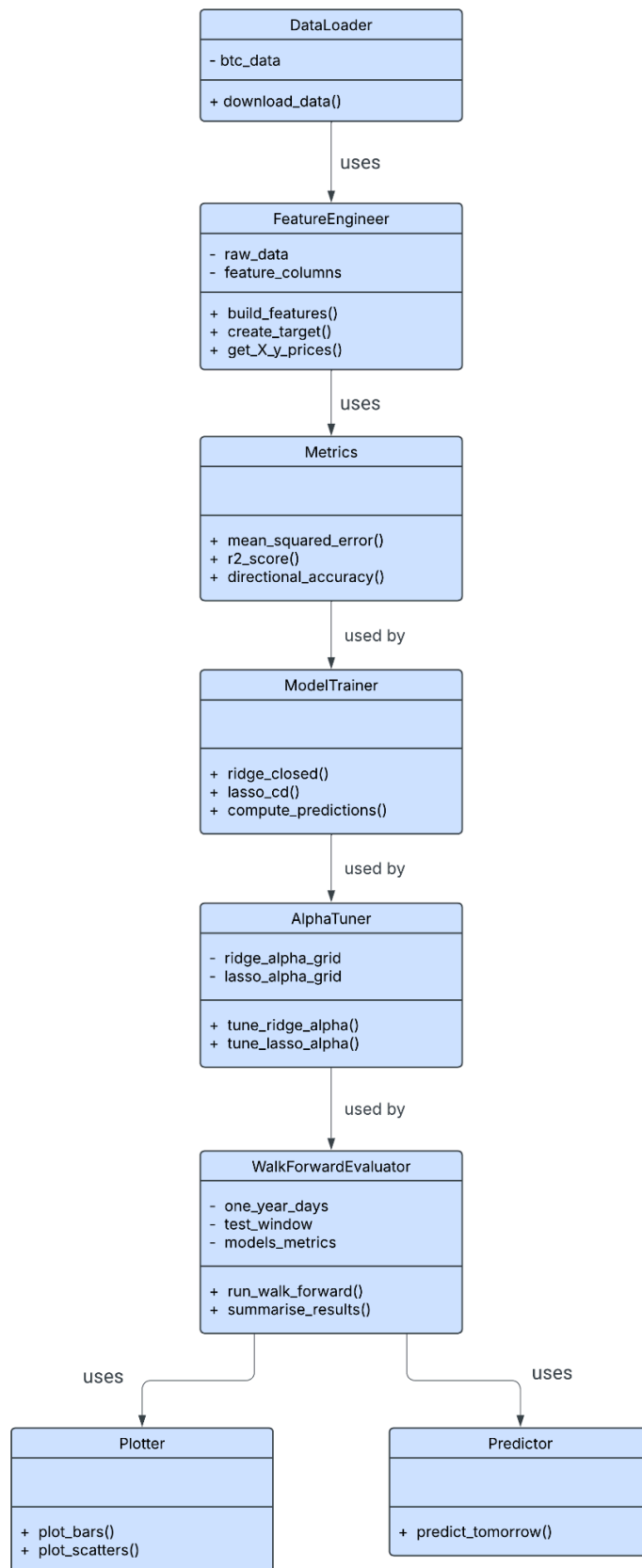
The use of manually implemented models rather than library equivalents ensures that the project meets the learning objectives of understanding regression methods at a fundamental level. It also provides flexibility for possible later extensions, such as integrating alternative loss functions or modifying regularisation schemes.

8.4 UML Architecture / Software Engineering

Although the current implementation is written procedurally within a single script, the final version of the project will adopt a fully object-oriented structure, as required by the module specification. The figure below presents a UML class diagram representing the planned modular architecture for the completed system.

This diagram does not reflect the present code structure directly. Instead, it illustrates how the system will be refactored into cohesive components such as DataLoader, FeatureEngineer, ModelTrainer, AlphaTuner and WalkForwardEvaluator. Each component encapsulates a specific responsibility currently implemented through functions in the procedural script.

The aim of this design is to ensure maintainability, separation of concerns, and easy extensibility especially for integrating additional algorithms (e.g., nearest neighbour, neural networks) and a graphical user interface in the final deliverable. The UML serves as the blueprint for transitioning the project from a prototype state to a full software-engineering implementation.



The UML diagram outlines how the current procedural system will be restructured into a set of classes for the final implementation. At present, all functionalities including data acquisition, feature construction, regression algorithms, metric calculations, hyperparameter tuning and walk-forward evaluation are contained within a single script and implemented through standalone functions. However, each block of functionality already corresponds naturally to one of the modules shown in the UML.

For example, the feature-engineering operations (LagReturn1, MA20, RSI, MACD, etc.) will migrate into a FeatureEngineer class, while the custom implementations of Ridge, Lasso and OLS will become methods within a dedicated ModelTrainer class. Similarly, the automatic tuning logic for hyperparameters will be encapsulated inside an AlphaTuner class rather than embedded in the main script.

By transitioning to this object-oriented structure, the system will become more modular and easier to extend. This is especially important for the next development stages, which include adding a nearest-neighbour algorithm, incorporating a neural-network model, supporting user-selected cryptocurrency pairs, and building a graphical user interface. The UML diagram therefore represents both the intended architecture and a roadmap for the software-engineering evolution of the project.

In terms of software engineering practices, version control has been used throughout the project via GitHub to track development, and maintain a consistent code history. Numerical testing and validation were also incorporated during development by comparing the outputs of the custom Ridge, Lasso and OLS implementations with their scikit-learn equivalents to ensure correctness. These practices reflect the transition towards a more complete software development workflow, which will continue into the next project phase.

8.5 Walk Forward Expanding Window Approach

A key methodological decision was the use of walk-forward expanding-window validation. Cryptocurrency markets evolve over time, and traditional random train-test splits are inappropriate because they violate temporal order and introduce future information into the training process. The expanding-window approach avoids these issues by training each model on all available past data and testing it only on the next unseen day.

This framework provides a realistic approximation of how a trading system would operate in practice, where each day's prediction is based solely on information available up to that point. As the window grows with each step, the model benefits from additional historical data while retaining chronological integrity.

8.6 Hyperparameter Tuning

Ridge and Lasso introduce regularisation parameters that must be tuned carefully. Hyperparameters were evaluated within the walk-forward framework to ensure that tuning itself did not leak future information. Different values of λ were tested iteratively, and the parameter that produced the lowest mean squared error in forecasting over the preceding window was selected for the next day's prediction.

This process exposed an important insight: while extremely large λ values stabilize coefficients, they can also oversmooth the model, harming predictive accuracy. Conversely, very small λ values behave similarly to OLS and suffer from volatility. The tuning strategy was therefore essential in balancing stability with sensitivity to meaningful signals.

8.7 Prediction Procedure

For each step in the walk-forward cycle, the model produces a single next-day closing-price forecast based on the features available at the end of the current window. Predictions were collected sequentially to form a full time-series of forecasted prices, which could then be compared with actual market values. This approach ensures that performance metrics reflect the real-world use case of daily forecasting rather than in-sample curve-fitting.

8.8 Evaluation Metrics

Model performance was evaluated using both error-based and directional metrics. Mean Squared Error (MSE) and Mean Absolute Error (MAE) measured the magnitude of prediction errors, while directional accuracy measured whether the model correctly predicted the direction of price movement rather than its magnitude. Directional accuracy is particularly relevant in financial contexts, where predicting whether the price will rise or fall can be more valuable than predicting the exact price level.

While the detailed results will be discussed in the next section, these metrics provided a structured and quantitative assessment of model behaviour throughout development.

9. Results & Discussion

The BTC-USD regression models were evaluated using a walk-forward expanding-window approach, producing daily one-step-ahead predictions for each model. This framework allowed for a fair comparison of Ordinary Least Squares (OLS), Ridge Regression and Lasso Regression under realistic forecasting conditions.

Figures 1–3 show the relationship between true and predicted prices for Lasso, Ridge and OLS respectively. Each plot includes a reference line representing perfect prediction. All three models track the general price level accurately, demonstrating that linear regression with engineered features is capable of approximating the broad price evolution of Bitcoin.

Figure 1: Lasso — True vs Predicted Prices (Walk-forward)

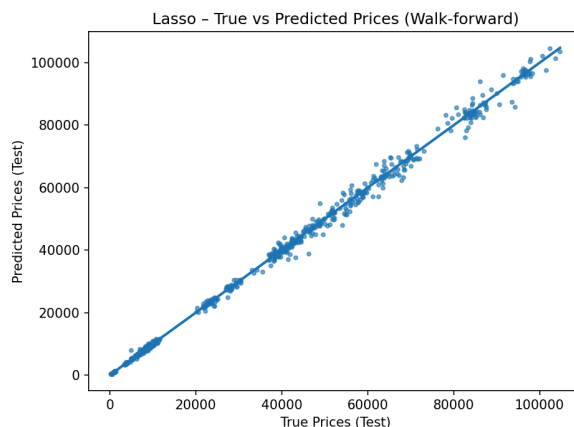


Figure 2: Ridge — True vs Predicted Prices (Walk-forward)

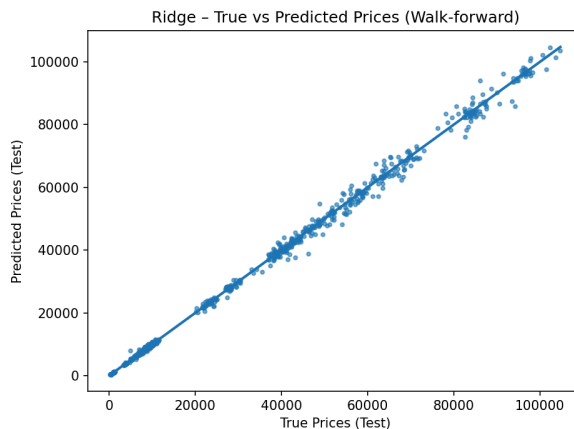
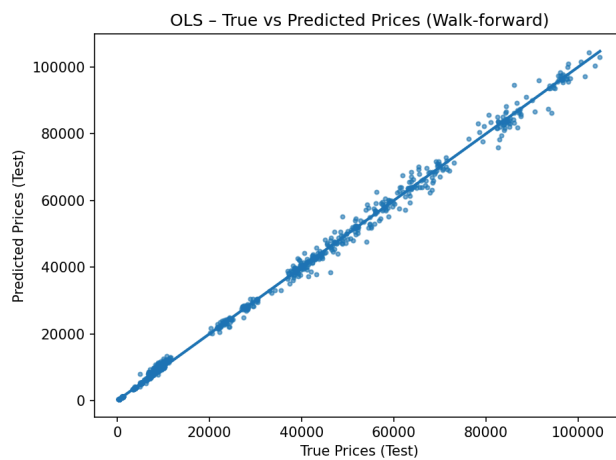


Figure 3: OLS — True vs Predicted Prices (Walk-forward)



Although all three models perform reasonably well, notable differences emerge in prediction stability. OLS shows the greatest dispersion around the diagonal line, confirming theoretical expectations that OLS is highly sensitive to noise and multicollinearity in the feature set. Ridge Regression reduces this instability, with points clustering more tightly around the diagonal. Lasso performs similarly to Ridge, suggesting that many features contribute minimally to predictive accuracy and can be suppressed without substantial performance loss.

Beyond price prediction, the models were also evaluated using returns, which provide a more stationary representation of market dynamics but are harder to predict due to Bitcoin's strong noise component.

Figures 4–6 show true versus predicted returns for Lasso, Ridge and OLS.

Figure 4: Lasso — True vs Predicted Returns (Walk-forward)

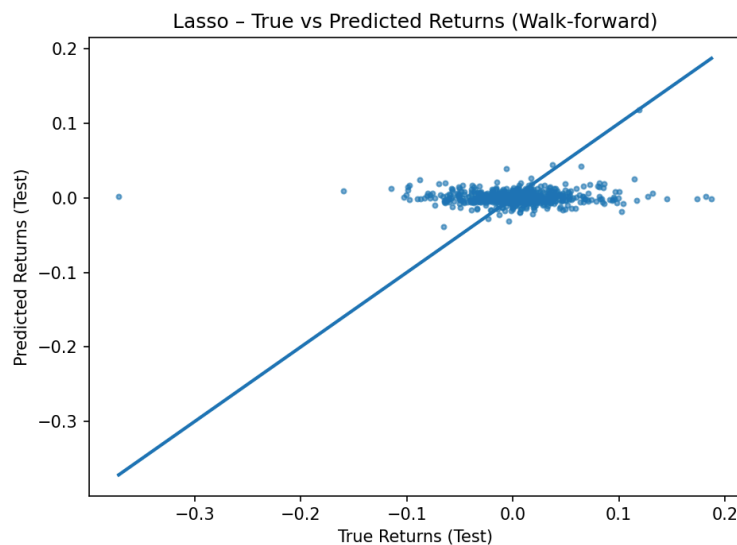


Figure 5: Ridge — True vs Predicted Returns (Walk-forward)

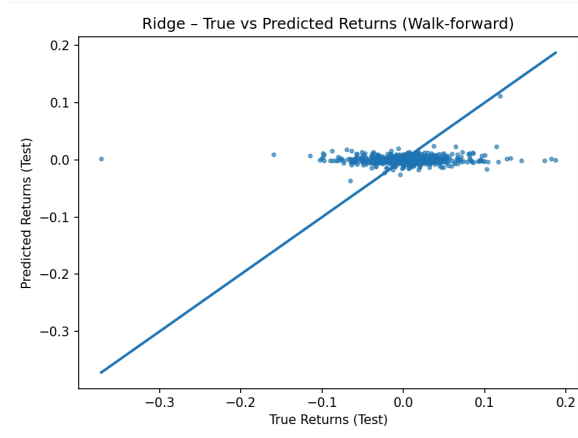
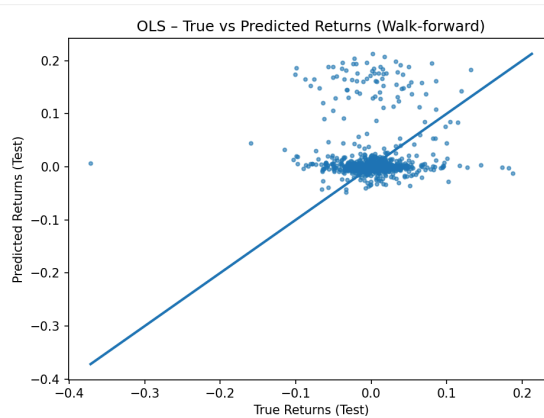


Figure 6: OLS — True vs Predicted Returns (Walk-forward)



As expected, return predictions exhibit far greater dispersion than price predictions. OLS again performed the worst, with widespread scatter and no tight concentration around the diagonal. Ridge and Lasso showed improved clustering, though returns remain difficult to model accurately. Importantly, both regularised models suppressed extreme prediction errors, indicating improved robustness.

To quantify comparative performance, Figures 7–9 present aggregated walk-forward metrics: directional accuracy, price MSE and return MSE.

Figure 7: Directional Accuracy by Model (Walk-forward)

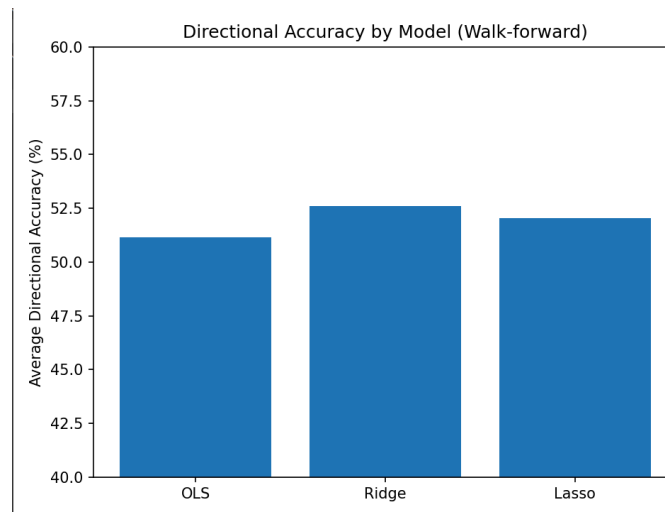


Figure 8: Average Price MSE by Model (Walk-forward)

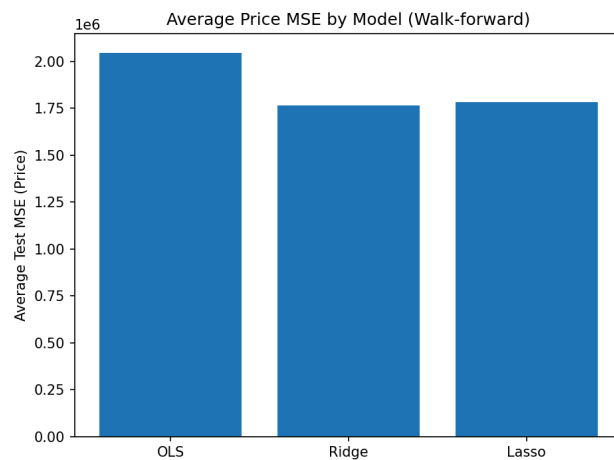
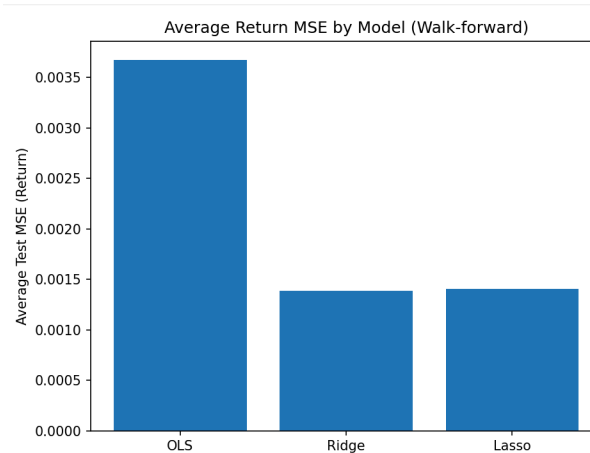


Figure 9: Average Return MSE by Model (Walk-forward)



Directional accuracy reveals meaningful differences. OLS achieves the lowest directional accuracy approximately 51%, performing close to random chance. Ridge achieves the highest around 52.5%, with Lasso slightly behind at around 52%. Although the differences appear small, they are significant in a financial context, where even minor directional improvements can lead to substantial long-term profitability. Also, these figures are in line with the expectation of fitting a linear model onto a financial dataset like BTC-USD whose movements are quite random.

Price MSE results further highlight the weaknesses of OLS. OLS exhibits the highest average MSE, indicating greater errors in predicting absolute price levels. Ridge and Lasso both achieve lower MSE scores, reinforcing the stabilising effect of regularisation.

Return MSE results mirror the directional-accuracy pattern: OLS performs worst, with error magnitudes far exceeding those of Ridge and Lasso. Ridge and Lasso achieve nearly identical performance, suggesting that both are effective at moderating the volatility inherent in return-series modelling.

It is also important to acknowledge the inherent limitations of the BTC-USD dataset itself. Bitcoin prices and returns show extremely high volatility, frequent structural breaks and strong noise components, all of which make short-term prediction particularly challenging for linear models. These characteristics naturally reduce the achievable accuracy ceiling and explain the wider dispersion observed in the return-prediction plots. As the project progresses, additional models such as nearest neighbours and neural networks may help capture nonlinearities that simple linear regression cannot fully represent.

Overall, the results consistently show that regularisation provides clear benefits in cryptocurrency forecasting. Ridge Regression offers the most balanced trade-off between bias and variance, leading to smoother predictions and better generalisation. Lasso performs similarly while introducing sparsity, which may prove useful in later feature-selection experiments. OLS serves as an important baseline but is too unstable for practical use in highly volatile markets like BTC-USD.

These findings validate the decision to use regularised linear models as the foundation of the forecasting pipeline and provide a strong empirical basis for extending the system with more advanced algorithms in the next development stage.

10. Progress to Date

The progress achieved so far aligns well with the expectations of an interim report. In weeks 3-6 as planned, the theoretical foundations of regression were clearly established, including an understanding of linear regression, Ridge Regression and Lasso Regression, as well as their respective advantages and limitations. Later on in week 8, A literature review was also completed, providing context for the use of regression models in financial prediction and highlighting the importance of feature engineering and sequential modelling techniques.

In week 7, A significant milestone accomplished was the development of a proof-of-concept model on synthetic data. This step validated the correctness of the custom regression implementations and confirmed their suitability for application to more complex datasets. The proof-of-concept also helped refine practical understanding of regularisation effects and debugging of the underlying matrix computations.

Between weeks 9-11, The most substantial achievement was accomplished which was the construction of a fully functioning BTC-USD regression pipeline. This system incorporates robust preprocessing, extensive feature engineering and an expanding-window prediction framework that mirrors real-world forecasting conditions. The manual implementation of OLS, Ridge and Lasso further demonstrates a deep engagement with the underlying algorithms rather than reliance on prebuilt libraries. Initial results show that Ridge and Lasso offer meaningful improvements over OLS, reinforcing the importance of regularisation in financial modelling.

The interim report therefore reflects a well-developed foundation, a working implementation and a clear direction for further progress. While the system is not yet complete, the work completed so far demonstrates strong momentum toward meeting the final requirements of the project.

11. Next Steps

The next stage of the project will involve expanding the modelling framework beyond linear regression. One planned extension is the integration of the nearest-neighbour algorithm, which offers a non-parametric approach based on similarity between historical feature vectors. This will allow comparison between linear regression methods and distance-based forecasting models, particularly in terms of local pattern sensitivity and robustness to noise.

Another major component of future work is the implementation of a neural-network-based model. Neural networks offer greater representational capacity than linear models and may capture non-linear relationships within Bitcoin price movements that regression cannot. This will also provide a valuable comparison between traditional machine-learning methods and modern deep-learning techniques.

A graphical user interface (GUI) is also planned for the final deliverable. The GUI will allow users to load cryptocurrency datasets, choose modelling options and visualise predictions interactively. Developing a user-friendly interface will require thoughtful software-engineering practices, modular code structure and clear separation between backend logic and frontend presentation.

The project will also be extended to allow forecasting for other cryptocurrency–USD trading pairs, providing a more versatile system and enabling comparative analysis across assets. Additional experimental variations such as sliding-window training, kernelised regression, feature-selection methods and integration of alternative indicators will be explored as time permits.

These future steps will build upon the existing regression pipeline and form the bridge between the interim achievements and the final project deliverables.

12. Conclusion

This interim report has presented the theoretical, practical and experimental progress achieved so far in developing a regression-based forecasting system for Bitcoin prices. The project began with the study of linear, Ridge and Lasso Regression, establishing a strong conceptual understanding of their behaviour, limitations and suitability for volatile financial data. A focused literature review demonstrated the persistent relevance of regression principles in modern financial-prediction research.

A proof-of-concept model using synthetic data verified the correctness of the custom regression implementations and provided insight into regularisation behaviour. Building on this foundation, a complete BTC-USD modelling pipeline was constructed, and early results show that regularised regression outperforms OLS in both stability and predictive quality. The project is now well positioned to expand into more advanced modelling techniques, including nearest-neighbour algorithms, neural networks and graphical user interfaces.

Overall, the work completed so far satisfies the expectations of an interim report and establishes a solid foundation for continued development. The next phase will focus on enhancing predictive performance, improving the usability of the system and exploring alternative modelling approaches to achieve the final project objectives.

13. Bibliography

1. Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.
2. Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221717310652>
3. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020) ‘Array programming with NumPy’, *Nature*, 585, pp. 357–362.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.
5. Hunter, J.D. (2007) ‘Matplotlib: A 2D Graphics Environment’, *Computing in Science & Engineering*, 9(3), pp. 90–95.
6. Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268.
<https://www.sciencedirect.com/science/article/abs/pii/S0957417414004473>
7. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Yahoo Finance (2024) Bitcoin USD (BTC-USD) Historical Data. Available at:
<https://finance.yahoo.com/quote/BTC-USD/history/>